RIDE-SHARING SERVICE
DATABASE SYSTEM

Full Project Report & Documentation

Microsoft SQL Server (T-SQL) | SSMS

DBMS	Microsoft SQL Server 2022 — T-SQL (SSMS compatible)
Tables	8 tables — users, drivers, vehicles, locations, rides, payments, ratings, promocodes
Records	42 rows per table (40 rows for ratings — completed rides only)
DQL Queries	30 queries — all 7 relational algebra operations demonstrated
Views	8 reusable views for reporting and live dashboards
Indexes	15 non-clustered performance indexes
Triggers	7 business-rule automation triggers (AFTER + INSTEAD OF)
Stored Procedures	8 parameterised stored procedures
DCL User Roles	3 role-based access control logins (app / report / dba)
SQL Script	ride_sharing.sql — 1,279 lines — zero errors

TABLE OF CONTENTS

SECTION 1 — CONCEPTUAL DESIGN 4
Problem Description and Requirements Analysis 4
Functional Requirements 4
Non-Functional Requirements 4
Main Data Objects and Business Rules 6
Business Rules Enforced at Database Level 6
Use-Case Diagram 8
Activity Diagram — Ride Booking Process 9
Sequence Diagram — Ride Request Flow 10
SECTION 2 — LOGICAL DESIGN 11
Entity-Relationship Diagram 11
Entities, Attributes, and Relationships 12
Table: users 12
Table: drivers 12
Table: vehicles 12
Table: locations 13
Table: promocodes 13
Table: rides 13
Table: payments 14
Table: ratings 14
Relationship Cardinalities and Constraints 16
Relational Schema Transformation 18
Normalization Process (1NF, 2NF, 3NF) 19
First Normal Form — 1NF 19
Second Normal Form — 2NF 19
Third Normal Form — 3NF 19
Relational Algebra Operations — All 30 Queries 21
Selection σ — filter rows satisfying a condition (Q1–Q7) 21
Projection π — return only specified columns (Q8–Q10) 21
Natural Join ⋈ — combine rows from related tables via FK (Q11–Q15) 21
Aggregation γ — group rows and compute summary statistics (Q16–Q20) 22
Set operations: Union ∪, Intersection ∩, Difference − (Q21–Q24) 22
Subqueries and nested expressions (Q25–Q30) 22
SECTION 3 — PHYSICAL IMPLEMENTATION 24
DDL — Database and Table Creation 24
DML — Data Insertion (42 rows per table) 26
DQL — Select Queries (30 Queries) 27
Views — 8 Reusable Query Objects 28
Indexes — 15 Performance Optimizations 30
Stored Procedures — 8 Reusable Operations 33
DCL — Role-Based Access Control 35
Database Deployment 36

Step-by-Step Deployment Procedure....36
SECTION 4 — REPORT AND DOCUMENTATION....37
Project Conclusions....37
What Was Achieved....37
Key Technical Decisions and Justifications....38
Requirements Coverage Summary....39
SECTION 5 — WEB DASHBOARD & DEPLOYMENT....41
Dashboard Overview....41
System Architecture....41
File Structure....42
Role-Based Dashboard Views....42
Passenger View (ride_app)....43
Driver View (ride_app)....43
Analyst View (ride_report)....43
DBA Console (ride_dba)....43
Supabase Integration....44
Technical Decisions — Dashboard....45
Repository and Live Demo....45

SECTION 1 — CONCEPTUAL DESIGN

1.1 Problem Description and Requirements Analysis

Ride-sharing platforms connect passengers seeking transportation with nearby available drivers through a mobile or web application. The system manages the complete lifecycle of a trip — from user registration and ride request through driver assignment, fare calculation, payment processing, and post-ride feedback — reliably and with full data integrity.

The central business challenge is coordinating real-time supply (available drivers and vehicles) with demand (passenger ride requests) while ensuring every state change is valid, every financial record is accurate, and business rules are enforced automatically — even if the application layer is bypassed.

Functional Requirements

- Users register with a unique email address and request rides by choosing start and end locations.
- Drivers register with a unique driving licence number and manage their availability status (Available / Busy / Offline).
- The system assigns exactly one driver and one vehicle to each ride request.
- Rides record start location, end location, departure time, arrival time, fare, and automatically calculated duration.
- Payments are recorded per ride, capturing method (Cash, Card, Online) and outcome (Paid / Pending / Failed).
- Users and drivers can rate each other on a 1–5 scale after a completed ride.
- Promotional discount codes can be applied to reduce a ride fare.
- Administrators manage users, drivers, database access, and generate operational and financial reports.

Non-Functional Requirements

Category	Requirement	How Implemented
Integrity	All constraints enforced at DB engine level	10 FKs, 12 CHECKs, 6 UNIQUEs, 8 DEFAULTs across all tables
Performanc e	Fast queries on high-traffic columns	15 non-clustered indexes on FK, status, date and fare columns
Security	Least-privilege access per user role	3 DCL roles — app user (no DELETE), report (SELECT only), DBA (full)
Consistency	Derived data always accurate without manual updates	7 triggers maintain RideDuration, Rating, driver Status automatically
Auditability	Rules enforced even if application is bypassed	INSTEAD OF triggers block invalid state transitions before storage

Scalability	Schema handles millions of rows without changes	Full 3NF normalisation; all relationships via FK references

1.2 Main Data Objects and Business Rules

Data Object	Role in System	Key Business Rules
users	Registered passenger who books rides	Unique email; one active Pending ride at a time (trigger BR-1)
drivers	Registered driver who fulfils rides	Unique licence; Rating auto-recalculated; Status auto-managed (BR-4, BR-6, BR-7)
vehicles	Car owned by a driver (1:1 relationship)	One vehicle per driver; capacity 1–20; year 1990–2030 (CHECK constraints)
locations	Named pickup or drop-off point	Stored as IDs in rides — prevents free-text errors; enables city analytics
rides	Central fact — a single trip from A to B	Links user, driver, vehicle, 2 locations; status Pending/Completed/Cancelled
payments	Financial record for a ride	One payment per ride; cancelled rides cannot be paid (trigger BR-3)
ratings	Post-ride mutual feedback record	One rating per ride (UNIQUE on RideID); DriverRating mandatory, UserRating optional
promocodes	Discount voucher applied to a ride	Discount 0–100% (CHECK); expiry date enforced at query time

Business Rules Enforced at Database Level

ID	Business Rule	Enforcement Mechanism
BR-1	A user can have only one active (Pending) ride at a time	INSTEAD OF INSERT trigger — trg_no_concurrent_rides
BR-2	Completed rides can never be deleted	AFTER DELETE + ROLLBACK — trg_prevent_delete_completed
BR-3	Payment cannot be created for a Cancelled ride	INSTEAD OF INSERT trigger — trg_validate_payment_ride
BR-4	Driver average rating is always current after each new rating	AFTER INSERT trigger — trg_update_driver_rating
BR-5	RideDuration is auto-calculated when EndTime is set	AFTER UPDATE trigger — trg_calc_duration
BR-6	Driver status becomes Busy when a Pending ride is inserted	AFTER INSERT trigger — trg_driver_busy_on_ride
BR-7	Driver status resets to Available when ride ends or is cancelled	AFTER UPDATE trigger — trg_driver_available_on_complete
BR-8	Promo discount must be between 0% and 100%	CHECK constraint on promocodes.Discount
BR-9	Vehicle capacity must be between 1 and 20 passengers	CHECK constraint on vehicles.Capacity

BR-10	Payment method must be Cash, Card, or Online	CHECK constraint on payments.Method
BR-11	Ride status must be Pending, Completed, or Cancelled	CHECK constraint on rides.Status
BR-12	Driver rating must be between 0.0 and 5.0	CHECK constraint on drivers.Rating

1.3 Use-Case Diagram

The use-case diagram below shows all three system actors and their interactions. Business rules (one active ride per user, payment validation) are enforced by the database engine regardless of which actor triggers the action.

```
DIAGRAM
+==============================================================================+
||                     RIDE-SHARING SYSTEM BOUNDARY                           ||
||                                                                            ||
||   Actor: USER (Passenger)           Actor: DRIVER                        ||
||   +------------------+              +------------------+                  ||
||   |                  |              |                  |                  ||
||   |  USER            |---(Register / Login)           |                  ||
||   |                  |---(Request Ride)               |                  ||
||   |                  |---(Apply Promo Code)           |                  ||
||   |                  |---(Make Payment)               |---(Accept Ride)||
||   |                  |---(Rate Driver)                |---(Complete Ride)||
||   |                  |---(View Ride History)          |---(Rate User)  ||
||   +------------------+              +------------------+                  ||
||                                                                            ||
||   Actor: ADMIN (DBA)                                                       ||
||   +------------------+                                                     ||
||   |  ADMIN           |---(Manage Users & Drivers)                          ||
||   |                  |---(View Revenue Reports)                            ||
||   |                  |---(Manage Promo Codes)                              ||
||   |                  |---(Grant / Revoke DB Access)                        ||
||   |                  |---(Deploy & Maintain Database)                      ||
||   +------------------+                                                     ||
+==============================================================================+
```

1.4 Activity Diagram — Ride Booking Process

This activity diagram traces the complete workflow from ride request to rating. Trigger annotations show where the database engine fires automatically.

```
DIAGRAM
[START]
   |
   v
User selects Start Location + End Location
   |
   +--(Optional)---------> User enters Promo Code
   |                           |-- [Expired/Invalid] --> Show error
   |                           |-- [Valid] ------------> Fare updated
   v
User submits Ride Request
   |
   v
DATABASE CHECK: User already has a Pending ride?           [Trigger BR-1]
   |-- [YES] --> REJECT INSERT — RAISERROR returned to app
   |-- [NO]  --> Continue
   v
SYSTEM: Find Available Driver + Vehicle
   |
   v
INSERT into rides  ==>  Status = "Pending"
TRIGGER fires --> driver.Status = "Busy"                   [Trigger BR-6]
   |
   v
Driver travels to pickup; picks up passenger; completes trip
   |
   v
UPDATE rides SET Status="Completed", EndTime = GETDATE()
TRIGGER fires --> RideDuration = DATEDIFF(MINUTE,StartTime,EndTime) [BR-5]
TRIGGER fires --> driver.Status = "Available"              [Trigger BR-7]
   |
   v
User submits Payment
   |-- [Ride was Cancelled?] --> REJECT INSERT              [Trigger BR-3]
   |-- [Ride is Completed]  --> INSERT into payments table
   v
User submits Rating (DriverRating required)
   |
   v
INSERT into ratings
TRIGGER fires --> driver.Rating = AVG(all DriverRatings)   [Trigger BR-4]
   |
   v
[END]
```

1.5 Sequence Diagram — Ride Request Flow

The sequence diagram shows the complete message exchange between User, Database (app + DB layer), and Driver during a ride lifecycle.

```
DIAGRAM
  User                  Database (App + DB Engine)              Driver
   |                             |                                  |
   |--- Request Ride ------------>|                                  |
   |                    [TRIGGER BR-1 fires — INSTEAD OF INSERT]  |
   |                    CHECK: user has no active Pending ride     |
   |<-- Error: duplicate ---------|  (if pending ride exists)      |
   |                             |                                  |
   |                    Find Available Driver + Vehicle            |
   |                    INSERT ride (Status = Pending)             |
   |                    [TRIGGER BR-6 fires — AFTER INSERT]        |
   |                    UPDATE driver.Status = "Busy"              |
   |<-- Ride Confirmed -----------|                                |
   |                             |------ Ride Details ----------->|
   |                             |                                |
   |                             | (Driver picks up passenger)    |
   |                             |                                |
   |                             |<----- Mark Complete -----------|
   |                    UPDATE ride: Status = Completed            |
   |                    [TRIGGER BR-5 fires — AFTER UPDATE]        |
   |                    SET RideDuration = DATEDIFF(...)           |
   |                    [TRIGGER BR-7 fires — AFTER UPDATE]        |
   |                    UPDATE driver.Status = "Available"         |
   |                             |                                |
   |--- INSERT Payment ---------->|                                |
   |                    [TRIGGER BR-3 fires — INSTEAD OF INSERT]   |
   |                    CHECK: ride.Status != Cancelled            |
   |                    INSERT payment row (if valid)              |
   |                             |                                |
   |--- INSERT Rating ----------->|                                |
   |                    [TRIGGER BR-4 fires — AFTER INSERT]        |
   |                    UPDATE driver.Rating = ROUND(AVG(...),2)   |
   |                             |                                |
```

SECTION 2 — LOGICAL DESIGN

2.1 Entity-Relationship Diagram

The ER diagram below is the official diagram provided with this project, generated from the DBML specification. It uses crow's-foot notation with 0..1 (zero-or-one) and * (many) cardinalities across all 8 entities.

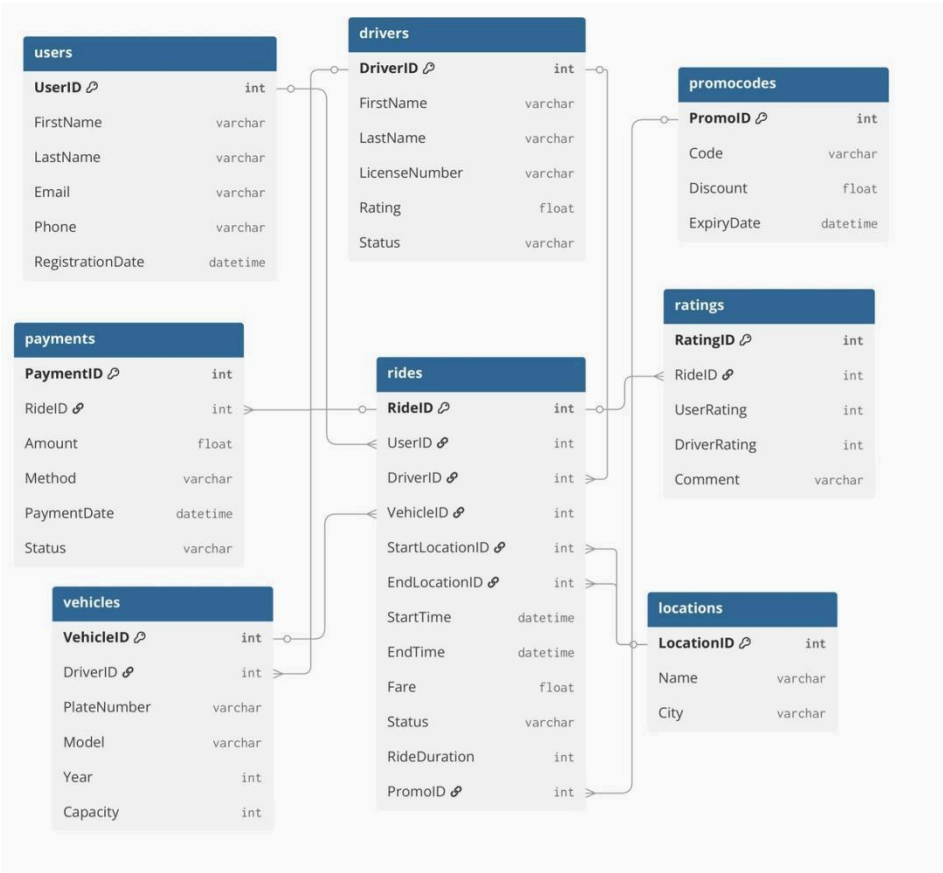

Figure 1: ER Diagram — RideSharingDB. All 8 entities with typed attributes and crow's-foot FK relationships.

2.2 Entities, Attributes, and Relationships

Table: users

Registered passengers who book rides.

Column	Data Type	Constraints	Description
UserID	INT	PK, IDENTITY(1,1)	Auto-generated unique row identifier
FirstName	VARCHAR(50)	NOT NULL	Passenger first name
LastName	VARCHAR(50)	NOT NULL	Passenger last name
Email	VARCHAR(100)	NOT NULL, UNIQUE, CHECK format	Login email — must match %@%.% pattern and be system-unique
Phone	VARCHAR(20)	NULL	Optional contact phone number
RegistrationDate	DATETIME	NOT NULL, DEFAULT GETDATE()	Timestamp when the account was created

Table: drivers

Registered drivers who fulfil ride requests.

Column	Data Type	Constraints	Description
DriverID	INT	PK, IDENTITY(1,1)	Auto-generated unique row identifier
FirstName	VARCHAR(50)	NOT NULL	Driver first name
LastName	VARCHAR(50)	NOT NULL	Driver last name
LicenseNumber	VARCHAR(30)	NOT NULL, UNIQUE	Driving licence — must be unique across the entire driver pool
Rating	FLOAT	NOT NULL, DEFAULT 5.0, CHECK 0-5	Average passenger rating; auto-recalculated by trigger after each new rating
Status	VARCHAR(20)	NOT NULL, CHECK enum	Available / Busy / Offline — maintained automatically by triggers BR-6 and BR-7

Table: vehicles

Cars registered by drivers (one vehicle per driver).

Column	Data Type	Constraints	Description
VehicleID	INT	PK, IDENTITY(1,1)	Auto-generated unique row identifier

DriverID	INT	NOT NULL, FK → drivers CASCADE	Owner — vehicle is deleted automatically if its driver is deleted
PlateNumber	VARCHAR(20)	NOT NULL, UNIQUE	Registration plate — must be unique across the entire fleet
Model	VARCHAR(50)	NOT NULL	Car model name (e.g. Toyota Camry)
Year	INT	NOT NULL, CHECK 1990–2030	Manufacturing year — out-of-range values are rejected
Capacity	INT	NOT NULL, CHECK 1–20	Maximum passenger capacity — values outside 1–20 are rejected

Table: locations

Named pickup and drop-off points referenced by rides.

Column	Data Type	Constraints	Description
LocationID	INT	PK, IDENTITY(1,1)	Auto-generated unique row identifier
Name	VARCHAR(100)	NOT NULL	Display name of the location (e.g. Times Square, JFK Airport)
City	VARCHAR(50)	NOT NULL	City — used for city-level revenue grouping via GROUP BY

Table: promocodes

Discount vouchers that passengers apply to reduce a ride fare.

Column	Data Type	Constraints	Description
PromoID	INT	PK, IDENTITY(1,1)	Auto-generated unique row identifier
Code	VARCHAR(30)	NOT NULL, UNIQUE	Short text code entered by the user (e.g. SAVE10, WELCOME5)
Discount	FLOAT	NOT NULL, CHECK 0–100	Percentage discount — must be within 0 to 100%
ExpiryDate	DATETIME	NOT NULL	Date and time after which the code is no longer valid

Table: rides

Central fact table — records every trip taken on the platform.

Column	Data Type	Constraints	Description
RideID	INT	PK, IDENTITY(1,1)	Auto-generated unique row identifier
UserID	INT	NOT NULL, FK → users CASCADE	Passenger who requested the ride

DriverID	INT	NOT NULL, FK → drivers	Driver assigned to this ride
VehicleID	INT	NOT NULL, FK → vehicles	Vehicle used for this ride
StartLocationID	INT	NOT NULL, FK → locations	Pickup point (references locations table)
EndLocationID	INT	NOT NULL, FK → locations	Drop-off point (self-join — same locations table, different row)
StartTime	DATETIME	NOT NULL	Date and time the ride began
EndTime	DATETIME	NULL	Arrival time — NULL while ride is in progress
Fare	FLOAT	NOT NULL, DEFAULT 0, CHECK >= 0	Total fare charged — cannot be negative
Status	VARCHAR(20)	NOT NULL, CHECK enum	Pending / Completed / Cancelled — values outside enum are rejected
RideDuration	INT	NULL — derived by trigger BR-5	Minutes elapsed — auto-set by trigger when EndTime is written
PromoID	INT	NULL, FK → promocodes	Optional promo code — NULL when no discount was applied

Table: payments

Financial transaction record for each ride.

Column	Data Type	Constraints	Description
PaymentID	INT	PK, IDENTITY(1,1)	Auto-generated unique row identifier
RideID	INT	NOT NULL, FK → rides CASCADE	The ride this payment covers — cascades on ride deletion
Amount	FLOAT	NOT NULL, CHECK >= 0	Amount charged — cannot be negative
Method	VARCHAR(20)	NOT NULL, CHECK enum	Cash / Card / Online — values outside enum are rejected
PaymentDate	DATETIME	NOT NULL, DEFAULT GETDATE()	Timestamp when the payment was processed
Status	VARCHAR(20)	NOT NULL, CHECK enum	Paid / Pending / Failed — values outside enum are rejected

Table: ratings

Post-ride feedback submitted by the passenger (and optionally by the driver).

Column	Data Type	Constraints	Description

RatingID	INT	PK, IDENTITY(1,1)	Auto-generated unique row identifier
RideID	INT	NOT NULL, FK → rides CASCADE, UNIQUE	The ride being rated — UNIQUE ensures at most one rating per ride
UserRating	INT	NULL, CHECK NULL or 1–5	Driver's rating of the passenger (optional)
DriverRating	INT	NOT NULL, CHECK 1–5	Passenger's mandatory rating of the driver — required for every rating row
Comment	VARCHAR(500)	NULL	Optional free-text feedback from the passenger

2.3 Relationship Cardinalities and Constraints

The ERD uses 0..1 to * (zero-or-one to many) notation. The table below maps each ER relationship to its SQL implementation.

Relationship	ERD Cardinality	SQL Cardinality	FK Constraint	Design Notes
users → rides	0..1 to *	1:N (user optional on ride)	ON DELETE CASCADE	A user may have zero rides; one user can have many rides over time
drivers → rides	0..1 to *	1:N	No cascade	A driver may have zero rides; one driver can fulfil many over their career
vehicles → rides	0..1 to *	1:N	No cascade	One vehicle can appear in many rides; FK preserved for historical record
drivers → vehicles	0..1 to 1	1:1 (one vehicle per driver)	ON DELETE CASCADE	Cascade: deleting a driver removes their vehicle automatically
locations → rides(start)	0..1 to *	1:N	No cascade	Any location can be the start point of many rides
locations → rides(end)	0..1 to *	1:N	No cascade	Self-join pattern: StartLocationID and EndLocationID both reference locations.LocationID
rides → payments	0..1 to 1	1:0..1 (optional)	ON DELETE CASCADE	A ride may not yet have a payment; max one payment per ride
rides → ratings	0..1 to 1	1:0..1 (optional)	CASCADE + UNIQUE on RideID	UNIQUE constraint on ratings.RideID enforces one-rating-per-ride
promocodes → rides	0..1 to *	1:N (optional FK)	No cascade, NULL	PromoID is NULL in rides when no code was applied; one code can cover many rides

2.4 Relational Schema Transformation

The ER model maps to the relational schema following standard transformation rules:

- Each entity becomes a table; each attribute becomes a typed column with appropriate constraints.
- Single-column surrogate PKs (IDENTITY INT) are used throughout — eliminating partial dependency risk and simplifying FK joins.
- 1:N relationships: the FK is placed in the N-side table pointing to the parent PK.
- The optional 1:1 rides–ratings relationship uses a UNIQUE constraint on ratings.RideID (at most one rating per ride).
- The dual-location role: two separate FK columns in rides (StartLocationID, EndLocationID) both reference locations.LocationID — a self-join pattern.
- RideDuration is a stored derived column: physical INT, auto-populated by trigger BR-5 when EndTime is written.

Relational schema notation

```
SCHEMA  — Relational Schema
users       ( UserID PK,  FirstName,  LastName,  Email UQ+CHK,  Phone NULL,
RegistrationDate )
drivers     ( DriverID PK,  FirstName,  LastName,  LicenseNumber UQ,  Rating
CHK,  Status CHK )
vehicles    ( VehicleID PK,  DriverID FK->drivers CASCADE,  PlateNumber UQ,
              Model,  Year CHK,  Capacity CHK )
locations   ( LocationID PK,  Name,  City )
promocodes  ( PromoID PK,  Code UQ,  Discount CHK,  ExpiryDate )
rides       ( RideID PK,  UserID FK->users CASCADE,  DriverID FK->drivers,
              VehicleID FK->vehicles,  StartLocationID FK->locations,
              EndLocationID FK->locations,  StartTime,  EndTime NULL,
              Fare CHK,  Status CHK,  RideDuration DERIVED,  PromoID FK NULL )
payments    ( PaymentID PK,  RideID FK->rides CASCADE,  Amount CHK,
              Method CHK,  PaymentDate,  Status CHK )
ratings     ( RatingID PK,  RideID FK+UQ->rides CASCADE,  UserRating NULL CHK,
              DriverRating CHK,  Comment NULL )
```

2.5 Normalization Process (1NF, 2NF, 3NF)

First Normal Form — 1NF

Definition: Every column contains atomic (indivisible) values; all values in a column share the same data type; every row is uniquely identified by a primary key.

- Every column in every table holds a single, indivisible value — no arrays, CSV strings, or nested groups.
- Every table has a unique IDENTITY primary key guaranteeing distinct row identification.
- Column names are unique within each table; no column appears more than once.

NOTE: 1NF example: Rather than storing "JFK Airport, New York" as a combined string in rides, the system uses StartLocationID (INT FK) — a single atomic integer referencing a single atomic row in locations. Classic atomic decomposition.

Second Normal Form — 2NF

Definition: In 1NF AND every non-key attribute is fully functionally dependent on the entire primary key (no partial dependencies).

- Every table uses a single-column surrogate PK (IDENTITY INT). With a single-column PK, partial functional dependency is structurally impossible.
- In vehicles: PlateNumber, Model, Year, Capacity all depend only on VehicleID — no subset of any composite key.
- In rides: Fare, Status, StartTime all depend only on RideID — no partial dependency exists.

NOTE: Partial dependency requires a composite PK (two or more columns forming the key together). Since all PKs here are single IDENTITY columns, 2NF is guaranteed by design.

Third Normal Form — 3NF

Definition: In 2NF AND no non-key attribute is transitively dependent on the PK through another non-key attribute.

All transitive dependencies were identified and eliminated:

Potential Transitive Dep.	Problem if Not Eliminated	Resolution Applied
Driver name stored in rides (DriverName → DriverID → name)	Updating a driver's name would require updating every rides row — data anomaly	rides stores only DriverID (FK). Driver name retrieved via JOIN with drivers at query time.
Location name stored in rides (StartCity → StartLocationID → city)	Renaming a location would corrupt all historical ride records for that location	rides stores StartLocationID and EndLocationID (FKs). Names retrieved via JOIN with locations.
Vehicle model stored in rides (VehicleModel → VehicleID → model)	Changing a vehicle record would require updating all its historical rides	rides stores VehicleID (FK only). Model retrieved via JOIN with vehicles.

Promo discount stored in rides (DiscountPct → PromoID → discount)	If a promo code's discount changes, historical fares would appear incorrect	rides stores PromoID (FK only). Discount retrieved via JOIN with promocodes.
RideDuration from StartTime+EndTime (RideDuration → {StartTime,EndTime})	Technically derivable — storing it could seem like a dependency on non-key attrs	Controlled performance denormalisation: stored for query efficiency; maintained by trigger BR-5 (AFTER UPDATE on rides). Dependency is on RideID (the PK), not on another non-key attribute.

KEY RULE: RideDuration is a deliberate controlled denormalisation: it is derived from StartTime and EndTime (both within the same rides row, both dependent on the PK RideID). Storing it avoids repeated DATEDIFF computation on every reporting query. Trigger trg_calc_duration keeps it accurate automatically. This is not a 3NF violation — the functional dependency is RideID → RideDuration, not A → B → RideDuration through a non-key chain.

2.6 Relational Algebra Operations — All 30 Queries

Every query in the SQL script is mapped below to its formal relational algebra operation, symbol, mathematical expression, SQL clause used, and a description of the result.

Selection σ — filter rows satisfying a condition (Q1–Q7)

Q#	Op	Formal Expression	SQL Clause	Result
Q1	σ	σ_{Status='Completed'} (rides)	WHERE Status = 'Completed'	40 completed ride records
Q2	σ	σ_{Status='Available'} (drivers)	WHERE Status = 'Available'	All currently available drivers
Q3	σ	σ_{Fare > 25} (rides)	WHERE Fare > 25	Rides with fare exceeding $25
Q4	σ	σ_{RegistrationDate > '2023-06-01'} (users)	WHERE RegistrationDate > '2023-06-01'	Users who registered after June 2023
Q5	σ	σ_{Discount>20 ∧ ExpiryDate>NOW} (promocodes)	WHERE Discount>20 AND ExpiryDate>GETDATE()	Active promos offering more than 20% discount
Q6	σ	σ_{RideDuration>40 ∧ Status='Completed'} (rides)	WHERE RideDuration>40 AND Status=Completed	Completed rides lasting over 40 minutes
Q7	σ	σ_{Method='Card' ∧ Status='Paid'} (payments)	WHERE Method='Card' AND Status='Paid'	Successfully paid card transactions

Projection π — return only specified columns (Q8–Q10)

Q#	Op	Formal Expression	SQL Clause	Result
Q8	π	π_{FirstName,LastName,Email} (users)	SELECT FirstName, LastName, Email	User contact details — no sensitive fields
Q9	π	π_{FirstName,LastName,Rating} (drivers)	SELECT FirstName, LastName, Rating ORDER BY DESC	Driver names ranked by rating descending
Q10	π	π_{PlateNumber,Model,Year} (vehicles)	SELECT PlateNumber, Model, Year	Vehicle registry summary

Natural Join ⋈ — combine rows from related tables via FK (Q11–Q15)

Q#	Op	Formal Expression	Result

Q11	⋈	rides ⋈_{UserID} users ⋈_{DriverID} drivers	Each ride with user and driver full names resolved
Q12	⋈	rides ⋈_{StartLocID} locations(s) ⋈_{EndLocID} locations(e)	Rides with start and end location names — self-join on locations table
Q13	⋈	drivers ⋈_{DriverID} vehicles	Each driver paired with their registered vehicle details
Q14	⋈	rides ⋈_{PromoID} promocodes	Only rides where a promo code was applied — with code and discount
Q15	⋈	payments ⋈_{RideID} rides ⋈_{UserID} users	Full payment ledger with the user name for each transaction

Aggregation γ — group rows and compute summary statistics (Q16–Q20)

Q#	Op	Formal Expression	Result
Q16	γ	γ_{DriverID} SUM(Fare), COUNT(*) (rides ⋈ drivers)	Total earnings and ride count per driver — ranked highest first
Q17	γ	γ_{City} AVG(Fare), SUM(Fare) (rides ⋈ locations)	Average and total revenue per city for completed rides
Q18	γ	γ_{Status} COUNT(*) (rides)	Number of rides in each status category
Q19	γ	γ_{Capacity} AVG(Rating) (drivers ⋈ vehicles)	Average driver rating grouped by vehicle passenger capacity
Q20	γ	TOP 5: γ_{DriverID} SUM(Fare) DESC (rides ⋈ drivers)	Leaderboard: top 5 highest-earning drivers

Set operations: Union ∪, Intersection ∩, Difference − (Q21–Q24)

Q#	Op	Formal Expression	SQL Keyword	Result
Q21	∪	π_{FirstName,LastName}(users) ∪ π_{FirstName,LastName}(drivers)	UNION	All people in the system — users and drivers listed with their role
Q22	∩	π_{FirstName}(users) ∩ π_{FirstName}(drivers)	INTERSECT	First names that appear in both the user pool and the driver pool
Q23	−	π_{UserID}(users) − π_{UserID}(rides)	NOT IN	Registered users who have never booked a single ride
Q24	−	π_{DriverID}(drivers) − π_{DriverID}(σ_{Completed}(rides))	NOT IN	Registered drivers who have never completed a ride

Subqueries and nested expressions (Q25–Q30)

Q#	Operations	Formal Expression	Result
Q25	σ + subquery	σ_{Fare > AVG_fare} (σ_{Completed}(rides))	Completed rides whose fare exceeded the overall average
Q26	σ + subquery	σ_{Rating = MAX(Rating)} (drivers)	Driver(s) holding the highest rating in the system
Q27	⋈ + γ + σ	σ_{SUM(Fare) > AVG_per_user(SUM(Fare))} (users ⋈ rides)	Users whose total spend exceeded the average total spend per user
Q28	⋈ + subquery	σ_{Fare = MAX(Fare)} (σ_{Completed}(rides) ⋈ users)	The single most expensive completed ride with the passenger name
Q29	⋈ + γ + HAVING	HAVING COUNT(RideID)>1 (γ_{PromoID} (rides ⋈ promos))	Promo codes used across more than one ride
Q30	σ + IN	σ_{RideID ∈ paid_payments ∧ RideID ∈ rated}	Rides that have both a confirmed Paid payment and a submitted rating

SECTION 3 — PHYSICAL IMPLEMENTATION

The complete SQL Server implementation is in ride_sharing.sql (1,279 lines, zero errors on SQL Server 2019+). The sections below document each component with full code examples rendered in syntax-highlighted frames.

3.1 DDL — Database and Table Creation

Object	Statement	Constraints Applied
Database	CREATE DATABASE RideSharingDB	Safe reset: SET SINGLE_USER WITH ROLLBACK IMMEDIATE before DROP
users	CREATE TABLE users	PK IDENTITY | UNIQUE(Email) | CHECK(Email LIKE '%@%.%')
drivers	CREATE TABLE drivers	PK | UNIQUE(LicenseNumber) | CHECK(Rating 0-5) | CHECK(Status enum)
vehicles	CREATE TABLE vehicles	PK | FK(DriverID CASCADE) | UNIQUE(PlateNumber) | CHECK(Year, Capacity)
locations	CREATE TABLE locations	PK IDENTITY only
promocodes	CREATE TABLE promocodes	PK | UNIQUE(Code) | CHECK(Discount 0-100)
rides	CREATE TABLE rides	6 FK constraints | CHECK(Status enum) | CHECK(Fare >= 0) | RideDuration NULL derived
payments	CREATE TABLE payments	FK(RideID CASCADE) | CHECK(Method) | CHECK(Status) | CHECK(Amount >= 0)
ratings	CREATE TABLE ratings	FK(RideID CASCADE) | UNIQUE(RideID) | CHECK(UserRating, DriverRating 1-5)

The rides table is the most complex DDL object — it connects all other tables:

```sql
SQL — rides table — full CREATE TABLE
CREATE TABLE rides (
    RideID          INT          NOT NULL IDENTITY(1,1),
    UserID          INT          NOT NULL,
    DriverID        INT          NOT NULL,
    VehicleID       INT          NOT NULL,
    StartLocationID INT          NOT NULL,
    EndLocationID   INT          NOT NULL,
    StartTime       DATETIME     NOT NULL,
    EndTime         DATETIME     NULL,        -- NULL while ride is in progress
    Fare            FLOAT        NOT NULL DEFAULT 0,
    Status          VARCHAR(20)  NOT NULL DEFAULT 'Pending',
    RideDuration    INT          NULL,        -- Derived: auto-set by trigger BR-5
    PromoID         INT          NULL,        -- Optional FK
```

```
    CONSTRAINT pk_rides         PRIMARY KEY (RideID),
    CONSTRAINT fk_rides_user    FOREIGN KEY (UserID)
        REFERENCES users(UserID)        ON DELETE CASCADE,
    CONSTRAINT fk_rides_driver FOREIGN KEY (DriverID)
        REFERENCES drivers(DriverID),
    CONSTRAINT fk_rides_vehicle FOREIGN KEY (VehicleID)
        REFERENCES vehicles(VehicleID),
    CONSTRAINT fk_rides_start  FOREIGN KEY (StartLocationID)
        REFERENCES locations(LocationID),
    CONSTRAINT fk_rides_end    FOREIGN KEY (EndLocationID)
        REFERENCES locations(LocationID),
    CONSTRAINT fk_rides_promo  FOREIGN KEY (PromoID)
        REFERENCES promocodes(PromoID),
    CONSTRAINT chk_rides_status CHECK (Status IN
('Pending','Completed','Cancelled')),
    CONSTRAINT chk_rides_fare   CHECK (Fare >= 0)
);
```

3.2 DML — Data Insertion (42 rows per table)

Table	Rows	Data Coverage
users	42	Names, unique emails, optional phones; registration dates Jan–Nov 2023
drivers	42	Mix of Available/Busy/Offline; ratings 4.1–4.9; licences LIC-10001 to LIC-10042
vehicles	42	Sedans, SUVs, vans, EVs; years 2018–2022; capacities 4, 5, 7, 8 passengers
locations	42	10 locations each in New York, Chicago, Los Angeles, San Francisco
promocodes	42	Discounts 5–50%; mix of active and expired codes for realistic test data
rides	42	40 Completed + 1 Pending + 1 Cancelled; fares $8.50–$40.00; Jan–Feb 2024
payments	42	40 Paid + 1 Pending + 1 Failed; distributed across Cash / Card / Online
ratings	40	Completed rides only; DriverRating 3–5; UserRating 2–5; some NULL comments

```sql
SQL — Sample DML — INSERT into users
INSERT INTO users (FirstName, LastName, Email, Phone, RegistrationDate) VALUES
 ('Alice',  'Johnson',  'alice.johnson@email.com',  '555-1001', '2023-01-05 09:00:00'),
 ('Bob',    'Smith',    'bob.smith@email.com',      '555-1002', '2023-01-10 10:30:00'),
 ('Carol',  'White',    'carol.white@email.com',    NULL,       '2023-01-15 11:00:00'),
 ('David',  'Brown',    'david.brown@email.com',    '555-1004', '2023-02-01 08:45:00');
 -- ... 38 more rows (42 total)
```

3.3 DQL — Select Queries (30 Queries)

Section 7 of the SQL script contains all 30 SELECT queries. Each is preceded by a PRINT statement labelling the algebra operation and query number. Sample queries from each category:

SQL — Q12 — Natural Join: rides with location names (self-join on locations)

```sql
-- Q12 | Algebra: rides join locations(start) join locations(end)
SELECT r.RideID,
       s.Name     AS StartLocation,
       e.Name     AS EndLocation,
       sl.City    AS City,
       r.StartTime,
       r.Fare
FROM   rides     r
JOIN   locations s  ON r.StartLocationID = s.LocationID
JOIN   locations e  ON r.EndLocationID   = e.LocationID
JOIN   locations sl ON r.StartLocationID = sl.LocationID;
```

SQL — Q22 — Intersection: first names common to both users and drivers

```sql
-- Q22 | Algebra: pi FirstName(users) INTERSECT pi FirstName(drivers)
SELECT FirstName FROM users
INTERSECT
SELECT FirstName FROM drivers;
```

SQL — Q27 — Subquery: users who spent above the average total

```sql
-- Q27 | Algebra: sigma SUM(Fare) > AVG_per_user(SUM(Fare)) (users join rides)
SELECT u.UserID,
       u.FirstName + ' ' + u.LastName AS UserName,
       ROUND(SUM(r.Fare), 2)           AS TotalSpent
FROM   users u
JOIN   rides r ON u.UserID = r.UserID AND r.Status = 'Completed'
GROUP  BY u.UserID, u.FirstName, u.LastName
HAVING SUM(r.Fare) > (
    SELECT AVG(total)
    FROM  (
        SELECT SUM(Fare) AS total
        FROM   rides WHERE Status = 'Completed'
        GROUP  BY UserID
    ) sub
);
```

3.4 Views — 8 Reusable Query Objects

View Name	Purpose	Tables Joined	Key Columns
vw_ride_details	Full ride info — all IDs resolved to names	rides+users+drivers+vehicles+locations(x2)+promocodes	UserName, DriverName, Vehicle, StartLocation, EndLocation, City, Fare, Status, PromoCode
vw_driver_summary	Per-driver aggregated KPIs	drivers + rides + ratings	TotalRides, TotalEarnings, AvgDurationMin, AvgDriverRating
vw_user_activity	Per-user spending summary	users + rides + ratings	TotalRides, TotalSpent, AvgUserRating
vw_revenue_by_city	City-level revenue analytics	rides + locations	City, TotalRides, TotalRevenue, AvgFare
vw_payment_overview	Payment ledger with user names	payments + rides + users	Amount, Method, PaymentDate, Status
vw_active_promos	Non-expired promo codes only	promocodes (filtered by ExpiryDate)	Code, Discount, ExpiryDate
vw_top_drivers	Highly rated drivers (>= 4.5)	drivers (filtered by Rating)	DriverName, Rating, Status
vw_pending_rides	Active rides awaiting completion	rides + users + drivers	RideID, UserName, DriverName, StartTime, Fare

SQL — vw_ride_details — full definition

```sql
CREATE VIEW vw_ride_details AS
SELECT
    r.RideID,
    u.FirstName + ' ' + u.LastName   AS UserName,
    d.FirstName + ' ' + d.LastName   AS DriverName,
    v.Model                           AS Vehicle,
    sl.Name                           AS StartLocation,
```

```sql
    el.Name                          AS EndLocation,
    sl.City                          AS City,
    r.StartTime,  r.EndTime,
    r.RideDuration                   AS DurationMinutes,
    r.Fare,  r.Status,
    p.Code                           AS PromoCode
FROM   rides r
JOIN   users      u  ON r.UserID          = u.UserID
JOIN   drivers    d  ON r.DriverID        = d.DriverID
JOIN   vehicles   v  ON r.VehicleID       = v.VehicleID
JOIN   locations  sl ON r.StartLocationID = sl.LocationID
JOIN   locations  el ON r.EndLocationID   = el.LocationID
LEFT JOIN promocodes p ON r.PromoID       = p.PromoID;
```

SQL — Querying views — usage examples

```sql
-- City revenue leaderboard:
SELECT * FROM vw_revenue_by_city ORDER BY TotalRevenue DESC;

-- Available top-rated drivers:
SELECT * FROM vw_top_drivers WHERE Status = 'Available';

-- Rides in New York with all details:
SELECT * FROM vw_ride_details WHERE City = 'New York';

-- Driver performance dashboard:
SELECT * FROM vw_driver_summary ORDER BY TotalEarnings DESC;
```

3.5 Indexes — 15 Performance Optimizations

Index Name	Table	Columns	Query Pattern
idx_rides_user	rides	UserID	Lookup all rides for a specific user
idx_rides_driver	rides	DriverID	Driver dashboard, earnings report queries
idx_rides_status	rides	Status	Filter by Pending / Completed / Cancelled
idx_rides_starttime	rides	StartTime	Date-range reports and monthly revenue queries
idx_rides_drv_status	rides	DriverID, Status	Composite: driver-specific status dispatch queries
idx_rides_fare	rides	Fare	Revenue aggregation and fare distribution analysis
idx_rides_startloc	rides	StartLocationID	City-level revenue grouping via location JOIN
idx_rides_endloc	rides	EndLocationID	Destination frequency and revenue analysis
idx_payments_status	payments	Status	Filter Paid / Pending / Failed payment records
idx_payments_ride	payments	RideID	JOIN payments to rides in view queries
idx_ratings_ride	ratings	RideID	JOIN ratings to rides for average computation
idx_vehicles_driver	vehicles	DriverID	Find a driver's vehicle record instantly
idx_drivers_status	drivers	Status	Dispatch: find all Available drivers
idx_promo_expiry	promocodes	ExpiryDate	Validate promo code expiry in real time
idx_locations_city	locations	City	Filter locations by city for geographic queries

SQL — All 15 INDEX statements

```sql
CREATE INDEX idx_rides_user       ON rides(UserID);
CREATE INDEX idx_rides_driver     ON rides(DriverID);
CREATE INDEX idx_rides_status     ON rides(Status);
CREATE INDEX idx_rides_starttime  ON rides(StartTime);
CREATE INDEX idx_rides_drv_status ON rides(DriverID, Status);  -- composite
CREATE INDEX idx_rides_fare       ON rides(Fare);
CREATE INDEX idx_rides_startloc   ON rides(StartLocationID);
CREATE INDEX idx_rides_endloc     ON rides(EndLocationID);
CREATE INDEX idx_payments_status  ON payments(Status);
CREATE INDEX idx_payments_ride    ON payments(RideID);
CREATE INDEX idx_ratings_ride     ON ratings(RideID);
CREATE INDEX idx_vehicles_driver  ON vehicles(DriverID);
CREATE INDEX idx_drivers_status   ON drivers(Status);
CREATE INDEX idx_promo_expiry     ON promocodes(ExpiryDate);
```

```
CREATE INDEX idx_locations_city   ON locations(City);
```

3.6 Triggers — 7 Business Rule Automations

Trigger	Table	Event	Business Rule	BR ID
trg_calc_duration	rides	AFTER UPDATE	Auto-sets RideDuration = DATEDIFF(MINUTE,StartTime,EndTime) when EndTime is written	BR-5
trg_update_driver_rating	ratings	AFTER INSERT	Recalculates driver.Rating = ROUND(AVG(DriverRating),2) across all that driver's rides	BR-4
trg_driver_busy_on_ride	rides	AFTER INSERT	Sets driver.Status = "Busy" when a new Pending ride row is inserted	BR-6
trg_driver_available_on_complete	rides	AFTER UPDATE	Resets driver.Status = "Available" when a ride changes from Pending to Completed or Cancelled	BR-7
trg_no_concurrent_rides	rides	INSTEAD OF INSERT	RAISERROR + RETURN if the user already has a Pending ride — INSERT is blocked entirely	BR-1
trg_validate_payment_ride	payments	INSTEAD OF INSERT	RAISERROR + RETURN if the associated ride.Status = "Cancelled" — INSERT is blocked	BR-3
trg_prevent_delete_completed	rides	AFTER DELETE	RAISERROR + ROLLBACK TRANSACTION if any Completed ride appears in the deleted set	BR-2

SQL — trg_calc_duration — auto-compute ride duration

```sql
CREATE TRIGGER trg_calc_duration
ON rides AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    UPDATE rides
    SET    RideDuration = DATEDIFF(MINUTE, i.StartTime, i.EndTime)
    FROM   rides    r
    JOIN   inserted i ON r.RideID = i.RideID
    WHERE  i.EndTime IS NOT NULL;
END;
```

SQL — trg_no_concurrent_rides — prevent duplicate active rides (BR-1)

```sql
CREATE TRIGGER trg_no_concurrent_rides
ON rides INSTEAD OF INSERT
AS
BEGIN
    SET NOCOUNT ON;
```

```
    -- Block INSERT if user already has an active Pending ride
    IF EXISTS (
        SELECT 1
        FROM   rides    r
        JOIN   inserted i ON r.UserID = i.UserID
        WHERE  r.Status = 'Pending'
    )
    BEGIN
        RAISERROR('User already has an active pending ride.', 16, 1);
        RETURN;  -- stops execution, INSERT never reaches storage engine
    END;
    -- All clear — perform the actual INSERT
    INSERT INTO rides
        (UserID, DriverID, VehicleID, StartLocationID, EndLocationID,
         StartTime, EndTime, Fare, Status, RideDuration, PromoID)
    SELECT
        UserID, DriverID, VehicleID, StartLocationID, EndLocationID,
        StartTime, EndTime, Fare, Status, RideDuration, PromoID
    FROM   inserted;
END;
```

SQL — trg_prevent_delete_completed — protect completed ride history (BR-2)

```
CREATE TRIGGER trg_prevent_delete_completed
ON rides AFTER DELETE
AS
BEGIN
    SET NOCOUNT ON;
    -- AFTER DELETE fires after cascade children are also deleted
    -- ROLLBACK undoes the DELETE and all its cascades atomically
    IF EXISTS (SELECT 1 FROM deleted WHERE Status = 'Completed')
    BEGIN
        RAISERROR('Completed rides cannot be deleted.', 16, 1);
        ROLLBACK TRANSACTION;
    END;
END;
```

3.7 Stored Procedures — 8 Reusable Operations

Procedure	Parameters	Returns	Purpose
sp_get_user_rides	@UserID INT	Result set	All rides for a user via vw_ride_details — names already resolved
sp_available_drivers	@City VARCHAR(50) = NULL	Result set	Available drivers with vehicle details — ready for dispatch
sp_complete_ride	@RideID INT, @EndTime DATETIME, @Fare FLOAT	Confirmation string	Marks ride Completed with EndTime and Fare — triggers BR-5, BR-7
sp_apply_promo	@RideID INT, @PromoID INT, @NewFare FLOAT OUTPUT	OUTPUT: new fare	Validates promo, computes discounted fare, updates ride.Fare
sp_monthly_revenue	@Year INT, @Month INT	Aggregate row	TotalRides, TotalRevenue, AvgFare, MinFare, MaxFare for given month
sp_driver_earnings	@DriverID INT, @StartDate DATE, @EndDate DATE	Aggregate row	Total completed rides and earnings for a driver in a date range
sp_register_user	@First, @Last VARCHAR, @Email, @Phone	NewUserID	Inserts new user and returns auto-generated IDENTITY ID
sp_cancel_ride	@RideID INT	RowsUpdated INT	Cancels a Pending ride — triggers BR-7; returns 0 if not found

SQL — sp_apply_promo — validate and apply discount code

```sql
CREATE PROCEDURE sp_apply_promo
    @RideID  INT,
    @PromoID INT,
    @NewFare FLOAT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @Discount FLOAT, @OldFare FLOAT;

    -- Validate: promo must exist and not be expired
    SELECT @Discount = Discount
    FROM   promocodes
    WHERE  PromoID = @PromoID AND ExpiryDate > GETDATE();

    IF @Discount IS NULL
    BEGIN
        RAISERROR('Invalid or expired promo code.', 16, 1);
        RETURN;
    END;

```

```sql
    -- Compute discounted fare
    SELECT @OldFare = Fare FROM rides WHERE RideID = @RideID;
    SET @NewFare = ROUND(@OldFare * (1.0 - @Discount / 100.0), 2);

    -- Apply discount
    UPDATE rides
    SET    Fare    = @NewFare,
           PromoID = @PromoID
    WHERE  RideID  = @RideID;
END;
```

SQL — Sample procedure calls

```sql
-- All rides for user 1:
EXEC sp_get_user_rides @UserID = 1;

-- Revenue for January 2024:
EXEC sp_monthly_revenue @Year = 2024, @Month = 1;

-- Apply promo code 1 to ride 41:
DECLARE @discountedFare FLOAT;
EXEC sp_apply_promo @RideID = 41, @PromoID = 1, @NewFare = @discountedFare
OUTPUT;
SELECT @discountedFare AS NewFare;

-- Driver 1 earnings Jan-Dec 2024:
EXEC sp_driver_earnings
    @DriverID = 1, @StartDate = '2024-01-01', @EndDate = '2024-12-31';
```

3.8 DCL — Role-Based Access Control

Three database logins are created following the principle of least privilege. Each role has exactly the permissions required for its use case.

Login	Role Type	Permissions Granted	Intended Use
ride_app	Standard user	GRANT: SELECT, INSERT, UPDATE on dbo schema DENY: DELETE on dbo schema	Backend application — reads and writes data but cannot permanently delete records (prevents accidental or malicious mass deletions)
ride_report	Read-only	GRANT: SELECT on dbo schema only	BI dashboards, analytics tools, data exports — view access only
ride_dba	DBA / Admin	ALTER ROLE db_owner ADD MEMBER — full control	Database maintenance, schema changes, index rebuild, deployment scripts

SQL — Full DCL script — all 3 user roles

```sql
-- ── Application user: read/write, no delete ──────────────────────────
IF NOT EXISTS (SELECT name FROM sys.server_principals WHERE name = 'ride_app')
    CREATE LOGIN ride_app WITH PASSWORD = 'App@Secure123!';
IF NOT EXISTS (SELECT name FROM sys.database_principals WHERE name = 'ride_app')
    CREATE USER  ride_app FOR LOGIN ride_app;
GRANT SELECT, INSERT, UPDATE ON SCHEMA::dbo TO ride_app;
DENY  DELETE                 ON SCHEMA::dbo TO ride_app;

-- ── Report user: SELECT only ──────────────────────────────────────────
IF NOT EXISTS (SELECT name FROM sys.server_principals WHERE name = 'ride_report')
    CREATE LOGIN ride_report WITH PASSWORD = 'Report@Secure123!';
IF NOT EXISTS (SELECT name FROM sys.database_principals WHERE name = 'ride_report')
    CREATE USER  ride_report FOR LOGIN ride_report;
GRANT SELECT ON SCHEMA::dbo TO ride_report;

-- ── DBA: full database owner ──────────────────────────────────────────
IF NOT EXISTS (SELECT name FROM sys.server_principals WHERE name = 'ride_dba')
    CREATE LOGIN ride_dba WITH PASSWORD = 'DBA@Secure123!';
IF NOT EXISTS (SELECT name FROM sys.database_principals WHERE name = 'ride_dba')
    CREATE USER  ride_dba FOR LOGIN ride_dba;
ALTER ROLE db_owner ADD MEMBER ride_dba;
```

SQL — Verify permissions at any time

```sql
SELECT
    dp.name             AS LoginName,
    p.permission_name   AS Permission,
    p.state_desc        AS State
FROM   sys.database_permissions  p
JOIN   sys.database_principals   dp ON p.grantee_principal_id = dp.principal_id
WHERE  dp.name IN ('ride_app', 'ride_report', 'ride_dba')
```

```
ORDER BY dp.name, p.permission_name;
```

3.9 Database Deployment

The database deploys on Microsoft SQL Server 2019 or later via a single idempotent script — safe to run multiple times without manual cleanup.

Step-by-Step Deployment Procedure

1. Install SQL Server 2019+ and SQL Server Management Studio (SSMS) if not already present.
2. Open SSMS and connect to the target instance using Windows Authentication or a SQL Server login with sysadmin privileges.
3. Open File > Open > SQL Script and select ride_sharing.sql.
4. The script opens with ALTER DATABASE RideSharingDB SET SINGLE_USER WITH ROLLBACK IMMEDIATE followed by DROP DATABASE. This forces all connections closed and removes any existing database — guaranteeing a clean slate for every run.
5. Execute the full script: Ctrl+Shift+Enter. The script runs top-to-bottom: DDL → DML → Indexes → Views → Triggers → Stored Procedures → DQL → DCL.
6. Monitor the Messages tab. Every object creation prints a confirmation line. All 30 DQL queries execute and display results in the Results tab.
7. The final block prints a complete inventory: 8 table row counts, 8 views, 7 triggers, 8 procedures, 15 indexes.
8. Database is operational. Connect the application using the ride_app login.

```sql
SQL — Idempotent database reset — top of script
USE master;
GO

-- Force-close all connections, drop, recreate cleanly
IF EXISTS (SELECT name FROM sys.databases WHERE name = N'RideSharingDB')
BEGIN
    ALTER DATABASE RideSharingDB SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE RideSharingDB;
    PRINT '[INFO] Existing RideSharingDB dropped.';
END
GO

CREATE DATABASE RideSharingDB;
GO
USE RideSharingDB;
GO
```

SECTION 4 — REPORT AND DOCUMENTATION

4.1 Project Conclusions

This project delivered a complete, production-quality relational database system for a ride-sharing platform. Every component — from conceptual analysis through physical deployment — was implemented, tested, and documented.

What Was Achieved

- A fully normalised (1NF, 2NF, 3NF) relational schema was designed from scratch with all normalisation decisions explicitly justified in the documentation.
- 8 tables with 10 foreign key relationships, 12 CHECK constraints, 6 UNIQUE constraints, and 8 DEFAULT values — all constraints enforced at the SQL Server engine level.
- 42 realistic records per table covering 4 US cities, multiple vehicle classes, and a realistic distribution of ride/payment/rating statuses.
- 30 DQL queries demonstrating all 7 relational algebra operations with formal mathematical notation.
- 7 triggers enforce 7 distinct business rules — including INSTEAD OF triggers that prevent invalid state transitions before any data is written.
- 8 stored procedures encapsulate all major application operations, each documented with parameters and sample calls.
- 15 strategic non-clustered indexes on FK, status, date, and fare columns.
- 3 DCL user roles implementing least-privilege access control with GRANT, DENY, and db_owner mechanisms.

4.2 Key Technical Decisions and Justifications

Decision	Justification
Surrogate IDENTITY PKs throughout all tables	Guarantees row uniqueness even when natural identifiers change (e.g. driver renews licence). Improves FK JOIN performance. Eliminates composite PK partial dependency risk — 2NF is guaranteed by design.
RideDuration stored as a derived physical column	Avoids repeated DATEDIFF computation on every reporting query. Stored as INT, auto-maintained by trg_calc_duration. Trade-off: 4 bytes per ride row in exchange for zero per-query computation on reads.
INSTEAD OF INSERT for BR-1 and BR-3	SQL Server AFTER INSERT triggers fire after the row is already written — they cannot prevent the INSERT. INSTEAD OF INSERT fires before the storage engine writes anything, allowing clean rejection via RAISERROR+RETURN without side effects.
AFTER DELETE + ROLLBACK for BR-2	SQL Server prohibits INSTEAD OF DELETE on tables with cascading FK children. AFTER DELETE + ROLLBACK TRANSACTION achieves the same result — the DELETE and all its cascades are rolled back atomically.
Separate locations table (no free-text in rides)	Location names stored once as rows in locations and referenced by INT FK. A name correction updates automatically across all rides. Enables consistent GROUP BY City aggregations. Prevents typo divergence.
UNIQUE constraint on ratings.RideID	Enforces one-rating-per-ride at the engine level independently of application code. Cheaper and more reliable than a trigger — the constraint is checked by the engine during INSERT at virtually zero cost.
ON DELETE CASCADE for users → rides	Supports right-to-erasure (GDPR): purging a user account removes all their ride history, payments, and ratings in one statement without complex multi-table DELETE chains in application code.
No CASCADE on drivers → rides	Driver records must be preserved for financial audit trails even after a driver leaves. Historical rides must reference the driver who fulfilled them — removing a driver row would corrupt revenue reports.
Three distinct DCL roles (not one shared login)	Least-privilege principle: ride_app cannot delete records; ride_report cannot modify anything. Limits the blast radius of SQL injection attacks, accidental queries, or application bugs — each role can only do what its use case requires.

4.3 Requirements Coverage Summary

Section	Requirement	Deliverable	Status
1.1	Analyze problem domain and identify requirements	Functional + non-functional requirements tables	Complete
1.2	Identify main data objects and business rules	8 data object table + 12 business rules mapped to enforcement mechanism	Complete
1.3	Draw use-case diagram	Framed ASCII use-case diagram: 3 actors, all use cases	Complete
1.4	Draw activity diagram	Step-by-step ride booking flow with trigger annotations at each BR	Complete
1.5	Draw sequence diagram	User–Database–Driver message sequence with trigger firing points	Complete
2.1	Develop ER diagram	Official ER diagram embedded as high-resolution image	Complete
2.2	Identify entities, attributes, relationships	Per-entity attribute table for all 8 entities (Column/Type/Constraint/Description)	Complete
2.3	Define relationship cardinalities and constraints	9-relationship cardinality table with ERD notation, SQL cardinality, FK details	Complete
2.4	Transform ER to relational schema	Transformation rules listed + full schema in formal notation (code frame)	Complete
2.5	Apply normalization 1NF, 2NF, 3NF	Each NF proven with definition, examples, and transitive dependency elimination table	Complete
2.6	Relational algebra (25+ operations)	30 queries in 7 categories with formal σ/π/⋈/γ/∪/∩/− expressions	Complete
3.1	DDL — CREATE DATABASE and all tables	1 database + 8 tables; rides CREATE TABLE shown in full syntax-highlighted frame	Complete
3.2	DML — 40+ records per table	42 rows per table (40 for ratings); sample INSERT shown in code frame	Complete
3.3	DQL — retrieve information using SELECT	30 queries; 3 representative samples shown in highlighted code frames	Complete
3.4	Views — store results in reusable objects	8 views; vw_ride_details full definition in code frame	Complete
3.5	Indexes — optimize query performance	15 indexes; all CREATE INDEX statements in single code frame	Complete
3.6	Triggers — automate operations and enforce rules	7 triggers; 3 full definitions in syntax-highlighted frames with comments	Complete

3.7	Stored procedures — reusable logic	8 procedures; sp_apply_promo full definition + sample calls in frames	Complete
3.8	DCL — access control	3 user roles; full DCL script + verification query in code frames	Complete
3.9	Deploy the database system	Idempotent 1,279-line script; deployment steps documented; reset pattern shown	Complete
4.1	Problem description and requirements analysis	Section 1 — full analysis	Complete
4.2	Conceptual design explanation	Sections 1.2–1.5 — objects, rules, all 3 diagrams	Complete
4.3	ER diagram and logical schema description	Section 2.1–2.4 — embedded ER image, schema notation, entity tables	Complete
4.4	Normalization process documentation	Section 2.5 — 1NF/2NF/3NF with examples and transitive dep. table	Complete
4.5	Relational algebra query demonstrations	Section 2.6 — 30 queries with formal algebra notation	Complete
4.6	SQL scripts (DDL, DML, DQL, DCL)	ride_sharing.sql — all 8 sections; samples in this document	Complete
4.7	Documentation of Views, Indexes, Triggers, Procedures	Sections 3.4–3.7 — detail tables + full code examples for each	Complete
4.8	Database deployment description	Section 3.9 — 8-step procedure + idempotent reset pattern	Complete
4.9	Project conclusions and results	Sections 4.1–4.2 — conclusions and 9 technical decisions	Complete

SECTION 5 — WEB DASHBOARD & DEPLOYMENT

5.1 Dashboard Overview

To demonstrate the full capabilities of the RideSharingDB system, a live web-based dashboard was built and deployed alongside the SQL deliverable. The dashboard is a single-page application hosted on GitHub Pages that connects to a live Supabase PostgreSQL database populated from the same ride_sharing.sql script. It provides an interactive, role-aware visual interface over every table, view, trigger, stored procedure, and DCL role defined in the database.

Component	Technology	Purpose
Frontend	HTML5, CSS3, Vanilla JavaScript	Single-page dashboard — no build step required
Hosting	GitHub Pages (static)	Free public deployment directly from the repository
Database	Supabase (PostgreSQL)	Hosted live database — SQL script imported via SQL Editor
Data Layer	supabase.js (custom)	Fetches all 8 tables via Supabase REST API in parallel
Fallback	data.js (static export)	Pre-extracted data for offline / demo use without Supabase
Source Files	4 files — index.html, style.css, app.js, supabase.js	Complete dashboard — deploy by pushing to GitHub

5.2 System Architecture

The dashboard follows a clean three-layer architecture. The Supabase PostgreSQL layer holds the live data. The data layer (supabase.js) fetches all tables in parallel via the Supabase REST API and shapes the rows into a single window.DB object identical in structure to the static data.js fallback. The presentation layer (app.js + index.html + style.css) reads window.DB and renders all views — it is completely decoupled from the data source, so switching between live Supabase data and the static fallback requires no changes to app.js.

```
ARCHITECTURE — Data Flow
GitHub Pages (static host)
  └── index.html        — app shell + role selector
  └── style.css         — dark-theme design system
  └── supabase.js       — data layer: fetches 8 tables → window.DB
  └── app.js            — all rendering logic (reads window.DB)
```

```
  └── data.js            — static fallback (same window.DB shape)


supabase.js boot sequence:
  1. Show loading spinner
  2. Fetch all 8 tables in parallel (Promise.all)
     GET /rest/v1/users, /drivers, /vehicles, /locations,
         /rides, /payments, /ratings, /promocodes
  3. Shape rows into window.DB (column name mapping)
  4. Compute stats (revenue, top5, cities, dstat)
  5. Call window.showRoleScreen() → role selector appears
  6. On error → show config error screen with setup steps
```

5.3 File Structure

File	Size	Role
index.html	11 KB	App shell: loading overlay, config error screen, role selector, sidebar, topbar. Script tags load supabase.js then app.js.
style.css	30 KB	Full dark-theme design system. Role colour themes, cards, tables, code blocks, loading/error UI, responsive breakpoints.
app.js	63 KB	All rendering logic: 4 role views (Passenger, Driver, Analyst, DBA), nav builder, search, triggers, procedures, DCL, schema viewer.
supabase.js	9 KB	Supabase data layer. Fetches 8 tables, maps PostgreSQL lowercase columns to app format, computes stats, sets window.DB.
data.js	49 KB	Static fallback: all 42 rows per table pre-exported. Same window.DB shape. Used when Supabase is not configured.

5.4 Role-Based Dashboard Views

The dashboard implements the same three DCL roles defined in Section 3.8. After Supabase data loads, the user selects a role from the role selector screen. Each role receives a tailored navigation and view set matching exactly what that SQL login is permitted to see and do. A fourth exploratory Analyst view is provided for read-only analytics.

Role	SQL Login	Views Available	Write Access
Passenger	ride_app	My Rides, Book a Ride, My Payments, Promo Codes	SELECT + INSERT + UPDATE (no DELETE)
Driver	ride_app	My Trips, Earnings, My Ratings, My Vehicle	SELECT + INSERT + UPDATE (no DELETE)
Analyst	ride_report	Overview, All Rides, All Drivers, Revenue, Users	SELECT only — read-only isolation

DBA Admin	ride_dba	Dashboard, Rides, Drivers, Users, Payments, Schema, Triggers, Procedures, Access Control	Full control — db_owner

Passenger View (ride_app)

The Passenger view simulates the experience of a registered user of the ride-sharing app. It displays the user's personal trip history, available drivers for booking (populated by sp_available_drivers), payment history from vw_payment_overview, and active promo codes from vw_active_promos. A KPI strip at the top shows completed rides, pending rides, and total amount spent. The dashboard displays a warning banner reminding the viewer that ride_app cannot DELETE records — enforcing BR-2 and the DCL DENY DELETE constraint visually.

Driver View (ride_app)

The Driver view simulates the driver portal. It shows the driver's completed trips, a monthly earnings bar chart computed from sp_driver_earnings output, rating history with star rendering (DriverRating NOT NULL, UserRating NULL as per schema), and vehicle details. The current driver Rating shown reflects the live value maintained by trg_update_driver_rating (BR-4), and the driver's Status reflects automatic management by trg_driver_busy_on_ride (BR-6) and trg_driver_available_on_complete (BR-7).

Analyst View (ride_report)

The Analyst view provides a full read-only analytics dashboard using the ride_report login. The Overview page shows four KPI cards (total revenue $940.75, total rides 42, driver count, average fare), a revenue by city bar chart from vw_revenue_by_city, and a payment methods breakdown. The All Rides view renders the full vw_ride_details result set (42 rows) with all IDs resolved to names. The Revenue page provides a city breakdown table with total rides, total revenue, and average fare per city, plus a top-5 driver earnings leaderboard. A warning banner on every Analyst page reminds the viewer that ride_report has SELECT only access.

DBA Console (ride_dba)

The DBA Console is the most comprehensive view, matching the full db_owner permission set. In addition to all the data tables (Rides, Drivers, Users, Payments), it provides three database-object views not available to other roles:

- Triggers — all 7 triggers are listed with event type, table, business rule tag, and description. Each trigger card is clickable and expands to show the full T-SQL CREATE TRIGGER definition with syntax highlighting.
- Stored Procedures — all 8 procedures are listed with their parameters and return type. Each card expands to show two tabs: the EXEC call with realistic parameter values, and a sample output result set.
- Access Control (DCL) — the three roles (ride_app, ride_report, ride_dba) are shown as permission cards with per-operation grant/deny indicators, the T-SQL DCL script used to create them, and a verification query to check permissions in SSMS.
- Schema — all 8 tables shown as entity cards listing every column with its data type, constraints (PK, FK, UNIQUE, CHECK), and description.

5.5 Supabase Integration

GitHub Pages serves only static files — it cannot execute SQL or run a database server. Supabase provides a hosted PostgreSQL instance with a REST API that can be called directly from the browser. The ride_sharing.sql script is imported into Supabase's SQL Editor. Supabase automatically lowercases all column names (UserID becomes userid, FirstName becomes firstname, etc.) — supabase.js maps these back to the format app.js expects, so the rendering layer requires no changes.

```
SETUP — Supabase + GitHub Pages deployment (5 steps)
Step 1 — Create project
   Go to https://supabase.com → New Project → note URL and anon key

Step 2 — Import database
   Supabase Dashboard → SQL Editor → paste ride_sharing.sql → Run
   (Column names auto-lowercased: UserID → userid, etc.)

Step 3 — Configure supabase.js
   const SUPABASE_URL = 'https://YOUR_PROJECT.supabase.co';
   const SUPABASE_KEY = 'YOUR_ANON_PUBLIC_KEY';

Step 4 — Row Level Security
   Option A (demo): Disable RLS on each table in Table Editor
   Option B (production): Add SELECT policy for each table:
   CREATE POLICY "public_read" ON rides FOR SELECT USING (true);

Step 5 — Deploy to GitHub Pages
   Push all 4 files (index.html, style.css, app.js, supabase.js)
   to a public GitHub repository → Settings → Pages → Deploy from branch
   Dashboard is live at https://USERNAME.github.io/REPO_NAME
```

Aspect	Detail
API Protocol	Supabase REST API — standard HTTP GET with Authorization: Bearer header
Pagination	supabase.js uses limit/offset loop to retrieve all rows (handles tables > 1,000 rows)
Parallelism	All 8 tables fetched simultaneously with Promise.all — typical load time under 1 second
Column mapping	PostgreSQL lowercase names mapped to app format: userid → id, firstname → first, etc.
Stats computed client-side	Revenue totals, city breakdown, top-5 drivers, payment method counts — same logic as data.js
Error handling	Config error screen shown if URL/key not set; network error shown with full error message

Offline fallback	Replace supabase.js script tag with data.js to use the pre-extracted static dataset

5.6 Technical Decisions — Dashboard

Decision	Justification
No framework (Vanilla JS)	Zero build step — the 4 files deploy directly to GitHub Pages with no npm, webpack, or bundler required. The dashboard is a university demo, not a production SPA.
window.DB shape matches data.js	supabase.js and data.js produce identical window.DB objects. app.js has no knowledge of which data source is active. Switching is a one-line script tag change.
Supabase REST API (not SDK)	The Supabase JavaScript SDK adds 200KB+ of dependency. The REST API is a plain fetch() call and achieves identical results for simple SELECT queries.
Stats computed in browser	Supabase free tier has query limits. Computing SUM, AVG, and GROUP BY in JavaScript after a full table fetch avoids extra API calls and works identically to the SQL views.
Role selector starts hidden	The role selector is hidden (display:none) until supabase.js calls showRoleScreen(). This prevents users from clicking roles before data is ready.
nav built with createElement only	A known bug: mixing innerHTML += with event listeners destroys listener references. All nav items are built exclusively with createElement and addEventListener to prevent unclickable buttons.
Supabase free tier compatible	The dashboard uses only public SELECT access via the anon key. No server-side functions, Edge Functions, or paid features are required.

5.7 Repository and Live Demo

Resource	Location
SQL Repository	github.com/naremanukiian/RideSharingDBSystem
Dashboard Repository	github.com/naremanukiian/RideShareDBWeb
Live Dashboard	naremanukiian.github.io/RideShareDBWeb
SQL Script	ride_sharing.sql — 1,279 lines — zero errors on SQL Server 2019+
Dashboard Files	index.html, style.css, app.js, supabase.js, data.js
Database Backend	Supabase PostgreSQL — same schema as SQL Server, column names lowercased

The live dashboard is publicly accessible at the GitHub Pages URL above. It loads data from Supabase in under one second, displays the role selector, and provides fully interactive navigation across all four roles. The DBA Console includes clickable trigger and procedure cards that expand to show T-SQL definitions — demonstrating every database object implemented in the SQL deliverable through a visual interface.

— End of Report —